The QueryGraph Stack

Stack release 0.3.0-e934f20 · built 2026-07-04

The Definitive Guide to the Governed Semantic Lakehouse

Alexy Khrabrov and Slava Tykhonov

querygraph.ai

Contents

Executive Summary

The QueryGraph stack is a governed semantic lakehouse for agentic AI: a set of coordinated open-source components that let AI agents answer questions over enterprise data while *proving* what they did — which semantics they used, which policies allowed it, which sources they touched, and which they were denied. The stack's thesis is that the differentiating infrastructure for enterprise AI is not a bigger model or a longer context window, but a verifiable chain from question to answer: deterministic identity, dual policy gating, signed envelopes, canonical hashing, and lineage anchoring, all projected through open standards.

Five components make up the stack, each independently useful, released in coordinated, codenamed versions:

- **Grust** — a backend-neutral property-graph API for Rust with a GQL/Cypher read surface and a dozen storage backends. The semantic graph substrate.
- **TypeSec** — agentic AI security in Rust's type system: unforgeable capabilities, TypeDID signed agent envelopes, audit events. The security fabric.
- **LakeCat** — a Rust-native Apache Iceberg REST catalog that binds catalog state, governed Sail planning, TypeSec receipts, and Grust projection to the same table transitions. The catalog boundary.
- **Sail** (fork) — the Spark-compatible compute engine, extended with a Cypher graph-query surface compiled into its SQL frontend. The lakehouse engine.
- **QueryGraph** — the semantic layer itself, in Rust (`qg-rust`) and Python (`qg-python`): four semantic projections (Semantic Croissant, CDIF, W3C DID, ODRL) plus OSI business semantics, RBAC+ODRL dual gating, Ed25519-signed TypeDID envelopes verified across languages, OpenLineage audit with signed attestations, an HTTP `/v1` API, MCP servers in both languages, and an A2A agent card.

As of this guide, the current releases are **QueryGraph 0.3.0 "Goshawk"** (the interoperability release, in both languages), over **Grust 0.11.0 "Crab"**, **TypeSec 0.11.0 "Burano"**, and **LakeCat 0.2.1 "Lynx"**, with 0.4-line development already carrying the governed navigator loop, envelope authentication, and a dependency-free Rust MCP server.

Links

What	Where
Workspace meta-repo (map, review, this guide's home)	https://github.com/querygraph/querygraph
QueryGraph Rust (`qg-rust`)	https://github.com/querygraph/qg-rust
QueryGraph Python (`qg-python`)	https://github.com/querygraph/qg-python
Grust	https://github.com/querygraph/grust
TypeSec	https://github.com/querygraph/typesec
LakeCat	https://github.com/querygraph/lakecat
Sail upstream (forked with the Cypher extension)	https://github.com/lakehq/sail
Goshawk releases	https://github.com/querygraph/qg-rust/releases/tag/v0.3.0 · https://github.com/querygraph/qg-python/releases/tag/v0.3.0
The dedicated QueryGraph book	`qg-rust/docs/book`
Semantic Croissant (MLCommons Croissant)	https://mlcommons.org/croissant/
CDIF (CODATA Cross-Domain Interoperability Framework)	https://cdif.codata.org/
W3C DID / ODRL	https://www.w3.org/TR/did-core/ · https://www.w3.org/TR/odrl-model/
OSI (Open Semantic Interchange)	https://github.com/open-semantic-interchange
OpenLineage	https://openlineage.io/
Model Context Protocol	https://modelcontextprotocol.io/
Agent2Agent (A2A)	https://github.com/a2aproject/A2A

This guide is the stack-wide companion to the dedicated QueryGraph book (which walks the semantic layer itself as a textbook, layer by layer). Read this guide to understand how the five components fit together, what each guarantees to the others, and how to operate the whole; read the dedicated book for the deep treatment of the semantic projections and the governed agent story.

The Stack at a Glance

A question enters the stack at the top and evidence comes out at the bottom:

1. An agent (or a person, or another agent framework over MCP or `/v1`) asks a question.
2. QueryGraph searches the **semantic model** — OSI business terms, Croissant field semantics, ontology terms — for what the question means here.
3. The **rights layer** decides, per source, whether this principal may read it: RBAC *and* ODRL must both allow. Every decision, allow or deny, becomes a receipt.
4. SQL is planned only over allowed **Sail** sources; denied sources are named as off-limits.
5. Synthesis happens — deterministic, or via any LLM — and the answer travels in a **TypeDID envelope** signed with Ed25519 under a `did:key` verification method.
6. An **OpenLineage** event (spec-conformant, schema-validated) records the run, and an Ed25519 **attestation** anchors the event hash.

Each step is somebody's component. The semantic graph lives in Grust (and can be queried through Sail's Cypher extension). The envelope machinery and capability model are TypeSec. The tables

come from a lakehouse whose catalog transitions LakeCat governs. And QueryGraph is the layer that makes them one system with one evidence chain.

Versions and Codenames

Stack releases are coordinated by codename, each repo keeping SemVer independently:

Component	Version	Codename	Pool
QueryGraph (both languages)	0.3.0	Goshawk	birds of prey
Grust	0.11.0	Crab	—
TypeSec	0.11.0	Burano	Venetian islands
LakeCat	0.2.1	Lynx	wild cats

All four sibling repos carry post-checkpoint development (Grust's Full39075 GQL profile, TypeSec's agent-framework interop plane and enforcement proxy, LakeCat's Iceberg REST hardening, QueryGraph's 0.4-dev line); the checkpoint releases above are what the published stack guarantees.

Grust: The Property-Graph Substrate

Grust is a modern property-graph API for Rust with a deliberately plain core model — `Graph = nodes + edges`, nodes and edges carrying ids, labels, and typed properties — and a strict separation between graph construction and database query languages. Application code builds a `grust::Graph`; backend crates decide how to persist or query it.

The workspace ships a facade crate (`grust-graph`) over a core (model, builder, schema, traversal IR, `GraphStore` trait) and backends for, among others: deterministic in-memory storage, SurrealDB, HelixDB, FalkorDB, LadybugDB, LanceDB, PostgreSQL (generic tables, SQL/PGQ, pgGraph), Turso, and — the one QueryGraph leans on — **`grust-sail`**, a Sail SparkConnect backend that stores graphs as Spark DataFrames.

On the read side, `grust-cypher` provides a Cypher/GQL surface over any store. QueryGraph uses Grust twice: the semantic graph (datasets, fields, ontology terms, agents, policies as nodes and edges) loads into a Grust store, and the Sail fork compiles a Cypher extension *into* the engine so the same graph is queryable from Spark Connect sessions.

Since the Crab checkpoint, Grust's development line has been building the "Full39075" GQL profile — `CALL { … }` subqueries, table-valued functions, `shortestPath()`/`allShortestPaths()`, backend-native passthrough — with an executable portable read corpus, and atomic batch execution behind the transaction surface.

TypeSec: Security in the Type System

TypeSec encodes permissions as types. A `Capability<CanWrite, Report>` is an unforgeable proof: its constructor is crate-private, its permission and resource parameters are phantom types, and the only production path to one runs a policy check and emits an audit event. If your code does not hold the capability, the guarded function does not exist for you. Violations are compile errors, not runtime surprises.

On top of the capability core, TypeSec provides the **TypeDID** machinery QueryGraph uses for agent identity and messaging: Ed25519 DID keys derived deterministically from seeds, `did:key` documents, and signed, encrypted agent envelopes (`ed25519-x25519-chacha20` profile) whose audit-safe attestations expose who did what to which resource at which privacy level — without revealing the payload. QueryGraph wraps every agent request and response in these envelopes; the

supervised multi-agent story mints capabilities for delegation, sensitive reads, and summary aggregation.

TypeSec's post-Burano development line runs remarkably parallel to QueryGraph's own: an agent-framework interop plane that guards OpenAI, Anthropic, LangChain, and Pydantic-AI tool calls; an MCP dialect and `mcp-gate`, a deny-by-default MCP stdio proxy; signed decision receipts; decision logging and replay; a PyPI-ready Python package; and an OpenAI/Anthropic-compatible **enforcement proxy** — the "governed inference proxy" pattern, built at the security layer. QueryGraph plans to consume these at the next TypeSec release rather than duplicate them.

LakeCat: The Catalog Boundary

LakeCat is a Rust-native Apache Iceberg REST catalog and QueryGraph foundation. Standard Iceberg clients speak to it on ordinary REST catalog paths (`/catalog/v1`); underneath, it binds catalog state, governed Sail planning, TypeSec receipts, and Grust projection to the same accepted table transition, so the catalog's view of a table and the governance evidence about it can never drift apart.

For QueryGraph, LakeCat's signature surface is the **bootstrap bundle** at `/querygraph/v1/bootstrap`: a projection of live catalog tables into Croissant, CDIF, OSI, ODRL, OpenLineage, and a Grust-ready graph envelope. The wire format lives in a shared crate (`qglake-bundle`), so `qg-rust` verifies and imports bundles without copying formats. Scan planning routes through the Sail-facing engine, with point-in-time and append-only incremental scans producing Iceberg REST plan-task tokens from stable Sail metadata.

Since Lynx, LakeCat's line has hardened the Iceberg REST surface (spec-correct error model and namespace semantics), added fail-closed Iceberg v4 requirement validation, and recorded a clean release-candidate proof — a full local handoff harness that creates a fixture, plans through Sail, writes Turso catalog and Grust graph state, verifies replay and OpenLineage evidence, and runs QueryGraph's locked verify/import commands.

Sail and the Cypher Extension

Sail is the Spark-compatible lakehouse engine (from lakehq) that QueryGraph uses as its compute and storage substrate: typed Parquet tables registered in a Spark Connect-compatible server, queryable from PySpark, with the QueryGraph audit trail (`qg_audit`) living alongside the data.

The stack's fork adds a **Cypher graph-query extension** compiled into Sail itself — roughly 5,600 lines across the SQL parser (Cypher AST), analyzer (graph analysis), and plan resolver, reusing Grust's property-graph model and schema validation rather than reimplementing them. The result: the same semantic graph that QueryGraph loads through Grust is `MATCH`-able from any Spark Connect session, no separate graph database required.

QueryGraph in Rust: The Governed Semantic Layer

`qg-rust` is the reference implementation. Its layers:

- **Semantic projections.** Every dataset is described four ways: Semantic Croissant (ML-ready dataset metadata, JSON-LD), CDIF (discovery, access, and profile projection), W3C DID (deterministic `did:oyd` identity documents), and ODRL (permissions and prohibitions as policy). OSI models project business terms — datasets, fields, metrics with per-dialect SQL expressions, ontology terms — over Croissant fields and governed Sail columns.
- **Governance.** RBAC roles and ODRL policies gate together — both must allow. Decisions are receipts. TypeDID envelopes carry agent requests and responses with real signatures.

- **Lakehouse.** Loaders stream CSV/TSV/XLSX into typed Parquet in Sail, register views, and keep a manifest; the `dataverse-e2e` path takes live Dataverse datasets through the whole chain, optionally calling Ollama through a DID-encrypted prompt gateway.
- **Lineage.** OpenLineage COMPLETE events (spec-conformant UUIDv5 run ids, validated against the official 2-0-2 JSON Schema), canonical event hashing, Ed25519-anchored attestations, JSONL and Sail sinks.
- **Interfaces.** A CLI (navigator bundles, the QGLake story, lakehouse loading, verification); the `/v1` HTTP API (`serve`); an MCP stdio server (`mcp-serve`); the A2A agent card (`agent-card`, also served at `/.well-known/agent-card.json`); and `verify-envelope`, which verifies qg-python's Ed25519 envelopes with no shared state.

The **QGLake story** — the Resilience Desk — is the governed multi-agent narrative both books use: a supervisor delegates to compartmentalized specialists (finance, energy, mobility, climate-health, reference), a restricted-data broker returns a signed denial instead of raw rows, synthesis sees only signed summaries, and the whole run emits an OpenLineage event anchored by an Ed25519 attestation. It is deterministic by design: the golden baseline the live navigator loop is measured against.

QueryGraph in Python: The Pythonic Mirror

`qg-python` mirrors the same layers Pydantic-v2-first, and adds the ecosystem surfaces Python is for:

- **Real crypto** (`crypto` extra): Ed25519 signing with seed-derived keys (`sha256(seed)` as private key, exactly TypeSec's derivation), `did:key` verification methods, envelope and attestation verification. Without the extra, digests are labeled `unsigned:sha256:` — never mistakable for signatures.
- **The governed navigator loop** (`querygraph answer`): question → semantic search (names, synonyms, bigrams) → RBAC+ODRL receipts → SQL plans over allowed sources → synthesis via any `Callable[[str], str]` (`openai_compatible_llm` binds Ollama, vLLM, llama.cpp, LM Studio, OpenRouter) or the deterministic baseline → signed envelope + schema-valid OpenLineage + attestation.
- **MCP server** (`mcp` extra, `querygraph mcp-serve`): search, metric resolution with dialect fallback, access checks whose denials are receipts, bundle building, the story, envelope verification, and `answer_question`.
- **Framework adapters**: LangChain `StructuredTool` (sync and async), vendor-neutral `TypeDidAgent.to_tool_schema()` in OpenAI and Anthropic flavors, and `api_auth` for minting `/v1` envelope-auth headers.
- **Lakehouse helpers**: PySpark against Sail's Spark Connect endpoint, registering the loaded tables and audit views.

The Evidence Chain, End to End

The stack's product is the chain, not any single hop:

1. **Identity** is deterministic. Agents derive `did:oyd` documents from seeds; signing keys derive from the same seeds; the same seed yields the same identity in Rust and Python.
2. **Policy** is dual-gated. RBAC says the role may act on the resource; ODRL says the policy permits the action and nothing prohibits it. Only both together allow. Every decision is a receipt with a reason.
3. **Messages** are signed envelopes. Payloads are canonically hashed (`sort_keys`, compact, `ensure_ascii`); signatures cover a documented payload (`querygraph-typedid-signing-v1`); verification methods are `did:key` identifiers anyone can resolve.

4. **Lineage** is standard. Runs emit OpenLineage events with deterministic UUIDv5 run ids under a shared namespace, validated against the official schema — the same seed produces the same run id in either language.
5. **Anchoring** is cryptographic. Event hashes are signed into attestations; the audit trail lands in JSONL and in Sail tables next to the data.

A useful way to read the chain: *the answer used OSI metric X, resolved to Sail table Y, under capability C and `odrl:read`, emitting OpenLineage run R anchored by attestation A — and source Z was denied, with a receipt.* No mainstream agent framework offers that sentence.

Interoperability Surfaces

Goshawk's organizing principle: QueryGraph does not compete with agent frameworks — it is the governed data and semantics plane they plug into.

- **`/v1` HTTP API** (Rust, axum): health, navigator bundles, the story, envelope audit, a semantic-model registry (import OSI or Croissant, list, fetch, search), and `answer`. With `--require-auth`, governed routes demand a signed envelope in `x-qg-envelope`, action `invoke`, resource bound to the request path, payload bound to the body hash — no cross-route replay, no body swapping. Denials are 401 receipts that teach the contract.
- **MCP** in both languages: one server reaches Claude, LangChain, PydanticAI, LlamaIndex, CrewAI, and the OpenAI Agents SDK with zero per-framework code. The Rust implementation is dependency-free JSON-RPC over stdio; the Python one uses the official SDK and adds `qg://` resources.
- **A2A**: both languages publish the same Agent2Agent card (five skills, a TypeDID security scheme) at `/.well-known/agent-card.json` and via `agent-card`; the equivalence suite asserts the cards agree.
- **Tool schemas**: `to_tool_schema()` exports OpenAI- and Anthropic-flavor JSON-Schema tool definitions accepted by most runtimes and local servers.
- **LLM providers**: the loop binds any OpenAI-compatible endpoint; the Rust Ollama path wraps calls in DID-encrypted prompt-bound envelopes.

The Cross-Language Contract

Rust and Python are held equivalent by an executable contract (`qg-python/tests/test_rust_equivalence.py`), not by discipline:

- `navigator` bundles are byte-identical modulo timestamps.
- The QGLake stories agree on governance semantics: same specialist roster, the restricted broker (and only it) denied, attestation schemas field-for-field identical.
- Python signs an envelope; the Rust CLI verifies it and rejects a tampered copy with a non-zero exit. Rust mints envelopes from the same seeds and Python's key derivation matches exactly (a fixture pins the shared `did:key`).
- Both CLIs' OpenLineage events validate against the official schema; a fixture pins the shared UUIDv5 run-id derivation.
- Both agent cards publish identical skills, capabilities, and security schemes.
- A live test boots `qg-rust serve --require-auth`, posts with a Python-minted auth header (200), and without one (401 with receipt).

Operating the Stack

The workspace expects sibling checkouts:

```
~/src/
├── querygraph/          # meta-repo: README, FABLE-REVIEW-1, this guide's sources
│   ├── qg-rust/         # reference implementation (this repo)
│   ├── qg-python/       # Pythonic mirror
│   ├── sail/            # fork, branch `grust`, Cypher extension
│   └── semantic/        # research: Polaris SemanticModel, OSI round-trips
├── grust/               # path dependency of qg-rust
├── lakecat/             # path dependency of qg-rust
└── typesec/             # consumed from crates.io
```

Quick start:

```
# Rust: tests, story, server, MCP
cd qg-rust && cargo test
cargo run -- qglake-story --json
cargo run -- serve --port 8080 --require-auth
cargo run -- mcp-serve

# Python: everything on
cd qg-python && uv sync --extra all
uv run pytest                      # includes the cross-language contract
uv run querygraph answer --question "Where do fiscal and energy stress overlap?"
uv run querygraph mcp-serve --osi model.yaml
```

Both repos run GitHub Actions CI; qg-rust's assembles the sibling layout. Release engineering is disciplined: SemVer, codename pools, coordinated stack versions, CHANGELOGs and RELEASES logs per repo, versioned EPUB/PDF book artifacts, and GitHub releases with attached wheels.

Roadmap

The near line (0.4-dev, already in progress):

- the navigator loop maturing from deterministic baseline to live LLM runs under identical receipts, and its Rust parity;
- the remaining `/v1` surface (lineage event queries, audit verification, access explanation) behind envelope auth;
- adopting TypeSec's next release: the interop plane, `mcp-gate`, signed decision receipts, and the enforcement proxy at the security layer.

The wider arc, from the workspace review (FABLE-REVIEW-1 in the meta-repo): Polaris `SemanticModel` entities and `/navigator-bundle` projection (LakeCat first, then upstream), `OSIMetricFacet` upstreaming to OpenLineage, dbt MetricFlow and Cube importers into OSI, Hugging Face Croissant importers, an ADBC/Flight SQL path for lightweight Python querying, Merkle-batched attestations, ontology import, cross-node federation — and the benchmark that motivates it all: measuring how much a governed semantic layer improves agent accuracy over the same lakehouse.

Link Index

- Meta-repo: https://github.com/querygraph/querygraph
- qg-rust: https://github.com/querygraph/qg-rust · release https://github.com/querygraph/qg-rust/releases/tag/v0.3.0
- qg-python: https://github.com/querygraph/qg-python · release https://github.com/querygraph/qg-python/releases/tag/v0.3.0
- Grust: https://github.com/querygraph/grust
- TypeSec: https://github.com/querygraph/typesec

- LakeCat: https://github.com/querygraph/lakecat
- Sail upstream: https://github.com/lakehq/sail
- Standards: Croissant https://mlcommons.org/croissant/ · CDIF https://cdif.codata.org/ · DID https://www.w3.org/TR/did-core/ · ODRL https://www.w3.org/TR/odrl-model/ · OSI https://github.com/open-semantic-interchange · OpenLineage https://openlineage.io/ · MCP https://modelcontextprotocol.io/ · A2A https://github.com/a2aproject/A2A